SPRING BOOT

userController.java

```java
package com.example.finaljwt.controller;

import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.security.core.context.SecurityContext;
import org.springframework.security.core.context.SecurityContextHolder;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.PostMapping;
import org.springframework.web.bind.annotation.RequestBody;
import org.springframework.web.bind.annotation.RestController;

import com.example.finaljwt.jwt.JwtUtil;
import com.example.finaljwt.model.userModel;

@RestController
public class userController {


    @Autowired
    private JwtUtil jwtUtil;



    @GetMapping("/hello")
    public String sayHello(){

        System.out.println(
SecurityContextHolder.getContext().getAuthentication());

        return "Hello Spring boot";
    }

    @GetMapping("/data")
    public String sayData(){
        return "Hello Spring boot Data";
    }

    @PostMapping("/login")
    public String loginMethod(@RequestBody userModel model){

        String username = "pirithika";
        String password = "1234";
        System.out.println(model.getUsername());
        System.out.println(model.getPassword());
```

```
        if(username.equals(model.getUsername()) &&
password.equals(model.getPassword())){
            return jwtUtil.generateToken(model.getUsername());
        }
        return "Invalid credentials";


    }

}
```

userModel.java

```
package com.example.finaljwt.model;

public class userModel {

    private String username;
    private String password;


    public userModel() {
    }
    public userModel(String username, String password) {
        this.username = username;
        this.password = password;
    }
    public String getUsername() {
        return username;
    }
    public void setUsername(String username) {
        this.username = username;
    }
    public String getPassword() {
        return password;
    }
    public void setPassword(String password) {
        this.password = password;
    }




}
```

securityConfig.java

```
package com.example.finaljwt.securityConfig;

import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;

import
org.springframework.security.config.annotation.web.builders.HttpSecurity;
import org.springframework.security.config.http.SessionCreationPolicy;
import org.springframework.security.web.SecurityFilterChain;
import
org.springframework.security.web.authentication.UsernamePasswordAuthentication
Filter;

import com.example.finaljwt.jwt.JwtFilter;

@Configuration
public class securityConfig {

    @Autowired
    private JwtFilter jwtFilter;


    @Bean
    public SecurityFilterChain filterChain(HttpSecurity http){

        http.
        csrf(csrf-> csrf.disable())
        .authorizeHttpRequests(auth ->
auth.requestMatchers("/login").permitAll().anyRequest().authenticated())
        .sessionManagement(session ->
session.sessionCreationPolicy(SessionCreationPolicy.STATELESS));

        http.addFilterBefore(jwtFilter ,
UsernamePasswordAuthenticationFilter.class);

        return http.build();
    }

}
```

jwtUtil.java

```
package com.example.finaljwt.jwt;

import java.security.Key;
```

```java
import java.util.Date;

import org.springframework.context.annotation.Bean;
import org.springframework.stereotype.Component;

import io.jsonwebtoken.Jwts;
import io.jsonwebtoken.SignatureAlgorithm;
import io.jsonwebtoken.security.Keys;

@Component
public class JwtUtil {

    private final Key SECRET  =
Keys.hmacShaKeyFor("mySecurityKeymySecurityKeymySecurityKey".getBytes());


    public String generateToken(String username){
        return Jwts.builder()
                .setSubject(username)
                .setIssuedAt(new Date())
                .setExpiration(new Date(System.currentTimeMillis() + 1000 * 60
* 60))
                .signWith(SECRET,SignatureAlgorithm.HS256)
                .compact();
    }


    public String extractName(String token){
        return Jwts.parserBuilder()
            .setSigningKey(SECRET)
            .build()
            .parseClaimsJws(token)
            .getBody()
            .getSubject();
    }
}
```

jwtFilter.java

```java
package com.example.finaljwt.jwt;

import java.io.IOException;
import java.util.ArrayList;

import org.springframework.beans.factory.annotation.Autowired;
```

```java
import
org.springframework.security.authentication.UsernamePasswordAuthenticationToke
n;
import org.springframework.security.core.context.SecurityContextHolder;
import org.springframework.stereotype.Component;
import org.springframework.web.filter.OncePerRequestFilter;

import jakarta.servlet.FilterChain;
import jakarta.servlet.ServletException;
import jakarta.servlet.http.HttpServletRequest;
import jakarta.servlet.http.HttpServletResponse;

@Component
public class JwtFilter extends OncePerRequestFilter {

    @Autowired
    private JwtUtil jwtUtil;

    @Override
    protected void doFilterInternal(HttpServletRequest request,
HttpServletResponse response, FilterChain filterChain)
            throws ServletException, IOException {

            String header = request.getHeader("Authorization");

            if(header != null && header.startsWith("Bearer ")){
                String token = header.substring(7);

                String username = jwtUtil.extractName(token);

              UsernamePasswordAuthenticationToken auth = new
UsernamePasswordAuthenticationToken(username,null, new ArrayList<>());

            SecurityContextHolder.getContext().setAuthentication(auth);
            }

            filterChain.doFilter(request, response);
        }




    }
```

Pom.xml

```xml
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
    xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
https://maven.apache.org/xsd/maven-4.0.0.xsd">
    <modelVersion>4.0.0</modelVersion>
    <parent>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-parent</artifactId>
        <version>4.0.5</version>
        <relativePath/> <!-- lookup parent from repository -->
    </parent>
    <groupId>com.example</groupId>
    <artifactId>finaljwt</artifactId>
    <version>0.0.1-SNAPSHOT</version>
    <name>finaljwt</name>
    <description/>
    <url/>
    <licenses>
        <license/>
    </licenses>
    <developers>
        <developer/>
    </developers>
    <scm>
        <connection/>
        <developerConnection/>
        <tag/>
        <url/>
    </scm>
    <properties>
        <java.version>21</java.version>
    </properties>
    <dependencies>
        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-security</artifactId>
        </dependency>
        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-webmvc</artifactId>
        </dependency>

        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-security-test</artifactId>
            <scope>test</scope>
```

```
        </dependency>
        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-webmvc-test</artifactId>
            <scope>test</scope>
        </dependency>
        <dependency>
  <groupId>io.jsonwebtoken</groupId>
  <artifactId>jjwt-api</artifactId>
  <version>0.11.5</version>
</dependency>
        <dependency>
  <groupId>io.jsonwebtoken</groupId>
  <artifactId>jjwt-api</artifactId>
  <version>0.11.5</version>
</dependency>
<dependency>
  <groupId>io.jsonwebtoken</groupId>
  <artifactId>jjwt-impl</artifactId>
  <version>0.11.5</version>
  <scope>runtime</scope>
</dependency>
<dependency>
  <groupId>io.jsonwebtoken</groupId>
  <artifactId>jjwt-jackson</artifactId> <!-- or jjwt-gson -->
  <version>0.11.5</version>
  <scope>runtime</scope>
</dependency>

    </dependencies>

    <build>
        <plugins>
            <plugin>
                <groupId>org.springframework.boot</groupId>
                <artifactId>spring-boot-maven-plugin</artifactId>
            </plugin>
        </plugins>
    </build>

</project>
```

Theory Notes

1. Spring Security with JWT

- **Spring Security** provides authentication and authorization.
- **JWT (JSON Web Token)** is a stateless way to authenticate users.
- Flow:
    1. User logs in → server validates credentials.
    2. Server generates JWT → sends to client.
    3. Client stores token (usually in localStorage or sessionStorage).
    4. Client sends token in `Authorization: Bearer <token>` header for protected requests.
    5. Server validates token → grants access.

2. Key Components in Your Project

- **Controller (`userController`)** → defines endpoints (`/login`, `/hello`, `/data`).
- **Model (`userModel`)** → holds username and password.
- **SecurityConfig** → configures Spring Security (disable CSRF, stateless sessions, JWT filter).
- **JwtUtil** → generates and validates tokens.
- **JwtFilter** → intercepts requests, extracts token, sets authentication in `SecurityContextHolder`.

3. Security Concepts

- **CSRF disabled** → because JWT is stateless and not vulnerable to CSRF in the same way as session-based auth.
- **SessionCreationPolicy.STATELESS** → no server-side sessions, every request must carry JWT.
- **SecurityContextHolder** → stores authentication info for the current request.

4. Best Practices

- Store secret keys in `application.properties` or environment variables.
- Use **BCryptPasswordEncoder** for password hashing instead of plain text.
- Add **roles/authorities** to JWT claims for role-based access.
- Implement **refresh tokens** for long-lived sessions.
- Secure endpoints with `.antMatchers("/admin").hasRole("ADMIN")`.

📝 Steps to Follow (Project Workflow)

1. Set up dependenciesAdd Spring Security, Web, and JJWT libraries in pom.xml.
2. Configure SecurityDisable CSRF, set stateless sessions, and add JWT filter in SecurityConfig.
3. Create JWT utilityImplement methods to generate and validate tokens using a secret key.
4. Build login endpointValidate credentials, generate JWT, and return it to the client.
5. Implement filterIntercept requests, extract token, validate, and set authentication in SecurityContextHolder.
6. Test endpointsUse Postman or curl to login, get token, and access protected endpoints with Authorization header.